time a client connects to a server, a TLS handshake is performed in which the server authenticates to the client, to guarantee that the client connected to the server it wanted to connect to (and not to a server under an attacker's control). Then fresh cryptographic keys are negotiated for this session and used to encrypt and authenticate every request and response from then on. You'll look in depth at TLS and HTTPS in chapter 7.

3.4.1 Enabling HTTPS

Enabling HTTPS support in Spark is straightforward. First, you need to generate a *certificate* that the API will use to authenticate itself to its clients. We will cover TLS certificates in depth in chapter 7. When a client connects to your API it will use a URI that includes the hostname of the server the API is running on, for example `api.example.com`. The server must present a certificate, signed by a trusted certificate authority, that says that it really is the server for `api.example.com`. If an invalid certificate is presented, or it doesn't match the host that the client wanted to connect to, then the client will abort the connection. Without this step, the client might be tricked into connecting to the wrong server and then send its password or other confidential data to the imposter.

Because you're enabling HTTPS for development purposes only, you could use a *self-signed certificate*. In later chapters you will connect to the API directly in a web browser, so it is much easier to use a certificate signed by a local certificate authority. Most web browsers do not like self-signed certificates. A tool called `mkcert` (https://github.com/FiloSottile/mkcert) simplifies the process